Investigating and Mitigating Degree-Related Biases in Graph Convolutional Networks

Xianfeng Tang[†], Huaxiu Yao[†], Yiwei Sun[†], Yiqi Wang[‡], Jiliang Tang[‡]
Charu Aggarwal[§], Prasenjit Mitra[†], Suhang Wang[†*]
The Pennsylvania State University[†], Michigan State University[‡], IBM T.J. Watson, NY, USA[§]
{xut10,huy144,yus162,pum10, szw494}@psu.edu {wangy206,tangjili}@msu.edu charu@us.ibm.com

ABSTRACT

Graph Convolutional Networks (GCNs) show promising results for semi-supervised learning tasks on graphs, thus become favorable comparing with other approaches. Despite the remarkable success of GCNs, it is difficult to train GCNs with insufficient supervision. When labeled data are limited, the performance of GCNs becomes unsatisfying for low-degree nodes. While some prior work analyze successes and failures of GCNs on the entire model level, profiling GCNs on individual node level is still underexplored.

In this paper, we analyze GCNs in regard to the node degree distribution. From empirical observation to theoretical proof, we confirm that GCNs are biased towards nodes with larger degrees with higher accuracy on them, even if high-degree nodes are underrepresented in most graphs. We further develop a novel Self-Supervised-Learning Degree-Specific GCN (SL-DSGCN) that mitigate the degree-related biases of GCNs from model and data aspects. Firstly, we propose a degree-specific GCN layer that captures both discrepancies and similarities of nodes with different degrees, which reduces the inner model-aspect biases of GCNs caused by sharing the same parameters with all nodes. Secondly, we design a self-supervised-learning algorithm that creates pseudo labels with uncertainty scores on unlabeled nodes with a Bayesian neural network. Pseudo labels increase the chance of connecting to labeled neighbors for low-degree nodes, thus reducing the biases of GCNs from the data perspective. Uncertainty scores are further exploited to weight pseudo labels dynamically in the stochastic gradient descent for SL-DSGCN. Experiments on three benchmark datasets show SL-DSGCN not only outperforms state-of-the-art self-training/self-supervised-learning GCN methods, but also improves GCN accuracy dramatically for low-degree nodes.

ACM Reference Format:
Xianfeng Tang, Huaxiu Yao, Yiwei Sun, YiqiWang, Jiliang Tang and Charu Aggarwal, Prasenjit Mitra, Suhang Wang. 2020. Investigating and Mitigating Degree-Related Biases in Graph Convolutional Networks. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM'20), October 19–23, 2020, Virtual Event, Ireland.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3340531.3411872

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CIKM '20, October 19–23, 2020, Virtual Event, Ireland
© 2020 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-6859-9/20/10…$15.00
https://doi.org/10.1145/3340531.3411872

1 INTRODUCTION

Over last few years, Graph Convolutional Networks (GCNs) have benefited many real world applications across different domains, such as molecule design [37], financial fraud detection [29], traffic prediction [30, 38], and user behavior analysis [11, 18, 27]. One of the most important and challenging applications for GCNs is to classify nodes in a semi-supervised manner. In semi-supervised learning, GCNs recursively update the feature representation of each node by applying node-agnostic transformation parameters. The whole training process is supervised by a few labeled nodes. *

However, degree distributions of most real-world graphs (e.g., citation graphs, review graphs, etc.) are power-law [1, 6, 9]. While the degree of major nodes are relatively small, few nodes on the long-tail side can dominate the training/learning of GCNs (we refer to Figure 1 in the analysis section as examples). We argue the power-law distributed node degree could hurt the performance of GCNs. On the one hand, nodes on such a graph are not independent and identically distributed (*i.i.d*), thus the parameters of a GCN should not be shared by all nodes. As suggested by [19], nodes with various degrees play different roles in the graph. Taking social networks as an example, high-degree nodes are usually leaders with higher influence; while most low-degree ones are at the fringes of the network. Current GCNs with node-agnostic parameters overlook the complex relations and roles of nodes with different degrees. On the other hand, the non-*i.i.d* node degrees can hurt the message-passing mechanism of GCNs. In fact, the superior performance of GCNs relies on the information propagating from labeled nodes to unlabeled nodes [10]. Obviously, nodes with lower degrees are less likely to be connected to labeled neighbors, compared with high-degree ones. As a result, less information are passed to these low-degree nodes, resulting in unsatisfying or even poor prediction performance. Few literature have explored the effects of non-*i.i.d* node degrees on real-world graphs. Recently, Wu et al. [31] propose a multi-task learning framework for GCNs, where the degree information is encoded into learned node representations. However, simply incorporating the value of degree as an extra feature does not solve the potential biases of GCNs, and low-degree nodes still suffer from the insufficient supervisions.

Therefore, in this paper, we analyze the degree-related biases in GCNs thoroughly. First, we design a series of observational tests to validate our assumption: the performance of GCNs are not evenly distributed regarding node degrees, and GCNs are biased on low-degree nodes. We further prove that the training of GCNs are more sensitive to nodes with higher degrees using sensitivity analysis and influence functions in statistics [16, 35]. Inspired by the analytic results, we realize two challenges of addressing the degree-related

*Suhang Wang is the corresponding author.

biases in GCNs as follows: **(C1) How to capture the complex relation among nodes with different degrees?** We recognize three types of node relations including global shared relation, local intra-relation, and local inter-relation. The global shared relation captures the common property among all nodes in the whole graph (i.e., what GCNs already done); the local intra-relation describes the similarity of nodes with the same degree; and the local inter-relation further characterizes the interacted information from nodes with similar degrees, as they may behave likewise. Therefore, a sufficiently generalized and powerful degree-specific GCN is required, which not only balances the global generalization and local degree customization of different nodes, but also captures local relation among nodes with various degrees; and **(C2) How to provide effective and robust information to facilitate the learning of GCNs on low-degree nodes?** It is non-trivial to make accurate predictions with limited labeled neighbors, due to the biased information propagation. How to create sufficient supervisions for low-degree nodes is extremely challenging.

To address these challenges, in this paper, we propose a novel Self-Supervised-Learning Degree-Specific GCN (SL-DSGCN), which reduces the biases from non-*i.i.d* node degrees in conventional GCNs. In particular, we first design a degree-specific GNN layer, which considers both globally shared information and local relation among nodes with same degree value. A recurrent neural network (RNN) based parameter generator is designed for modeling the inter-degree relation, which is ignored in the prior work DEMO-Net [31]. We then leverage the massive unlabeled nodes to construct artificial supervisions for low-degree nodes. We propose a self-supervised-learning paradigm where a Bayesian neural network serves as the teacher and assigns pseudo/soft labels jointly with uncertainty scores on unlabeled nodes. We further utilize the uncertainty scores as a guidance in stochastic gradient descent to prevent overfitting inaccurate pseudo labels when training SL-DSGCN. SL-DSGCN is evaluated on three benchmark datasets and show superior performance over state-of-the-art methods. Besides, it reduces label prediction error on low-degree nodes dramatically.

In summary, our contributions are three-fold:

- We study a novel problem of addressing the degree-related biases in GCNs. To the best of our knowledge, we are the first to analyze this problem empirically and theoretically.
- We design SL-DSGCN that tackles the degree-related biases in GCNs from both model and data distribution aspects using the proposed degree-specific GCN layer and self-learning algorithm, correspondingly.
- We validate SL-DSGCN on three benchmark graph datasets and confirm that SL-DSGCN not only out-performs state-of-the-art baselines, but also improves the prediction accuracy on low-degree nodes significantly.

2 RELATED WORK

In this section, we review related works, which includes graph neural networks and self-supervised learning.

2.1 Graph Convolutional Neural Networks

Graph data are ubiquitous in real-world. Recently, graph convolutional neural networks (GCNNs) have achieved state-of-the-art performance for many graph mining tasks [10, 15, 34], and many efforts have been taken [13, 25, 26, 33, 35, 36]. In general, these GCNNs could be divided into two categorizes: spectral based GCNNs and spatial-based GCNNs. Bruna et al. firstly propose the spectral based GCNNs [4] by applying the spectral filter on the local spectral space according to the spectral graph theory. Following this work, various spectral-based GCNNs [3, 8, 10, 15] are developed to improve the performances. GCN [15] aggregates the neighborhood information from the perspective of spectral theory. With the similar intuition, GraphSAGE [10] extends prior works in the inductive setting. The spectral based GCNNs usually require to compute the Laplacian eigenvectors or the approximated eigenvalues as suggested by spectral theory, and these methods are inefficient on large scale graph. Different from the spectral based ones, to improve the efficiency, the spatial-based GCNNs [2, 28, 39] attempt to directly capture the spatial topological information and use the mini-batch training schema. For example, DCNN[2] combines graph convolutional operator with the diffusion process and Veličković et al. proposes the graph attention network [28] with the self-attention mechanism on the neighbors of the node and assign different weights accordingly during the aggregation process. Of all these GCNNs, GNNs [15] are highly favorable by the computer science community [17, 24] due to the reliable performance. Thus, we select GCNs for this work.

Though GCNs have show promising results, recent advancements [7, 34, 42] also reveal various issues of GCNs including the over-smoothing and the vulnerability. In this paper, we empirically validate a new issue of GCNNs, i.e., *GCNNs are biased towards high-degree nodes and have low accuracy on low-degree ones*. A potential reason is the imbalanced labeled node distribution. The issue is amplified when the total amount of labeled node for training is small.

2.2 Self-Supervised Learning

Recently, self-supervised learning, which generally refers to explicitly training models with automatically generated labels, has become a successful approach in computer vision and natural language processing for unsupervised pretraining and for addressing the issue of lacking labeled data [14]. For example, pretext tasks such as Image Inpainting [21] and Image Jigsaw Puzzle [20] are widely adopted in computer vision domains.

The success of self-supervision has motivated its study in graph mining domains. Though still in its early stage, there are a few seminal work trying to exploit self-supervised training to improve the performance of GCNs [12]. For example, Li et al. [17] propose the co-training and self-training based GCN models by expanding the training node set with pseudo labels from its nearest neighborhoods; Sun et al. [24] combine DeepCluster [5] with a multi-stage training framework so that the generalization performance of GCNs with few labeled nodes are improved.

Despite their initial success, existing studies mainly utilize self-supervised training as a trick for GCNs, without digging deep into why self-supervised training can improve the performance and what kind of nodes are benefited most from the self-supervised training. Our work is inherently different from existing ones on self-supervised GCNs. The lack of labeled neighborhoods among low-degree nodes motivate us to explore self-supervised training to balance the label distribution. The proposed self-supervision based one teacher-student network is also different from existing work. In addition, we also address the issue from the perspective of degree-specific layers.

To the best of our knowledge, only few work address the degree non-*i.i.d* sampled problem. DEMO-Net [31] learn the degree-specific representation for each node via the explicitly designed hash table. This work is significantly different from ours. Besides, *it fails to capture the similarity of nodes with close degree values*, where the RNN-based parameter generator in SL-DSGCN is able to do so.

3 PRELIMINARIES

We use $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X})$ to denote a graph, where $\mathcal{V} = \{v_1, \ldots, v_N\}$ is the set of N nodes, $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ represents the set of edges, and $\mathbf{X} = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$ indicates node features. We use $d_i \in \mathbb{R}^+$ to denote the degree of node v_i. In semi-supervised setting, partial nodes come with labels and are denoted as $\mathcal{V}^L$, where the corresponding label of node v_i is y_i. Similarly, the unlabeled part is defined as $\mathcal{V}^U$.

We introduce the architecture of a GCN. A GCN contains multiple layers. Each layer transforms its input node features to another Euclidean space as output. Different from fully-connected layers, a GCN layer takes first-order neighbors' information into consideration when transforming the feature vector of a node. This "message-passing" mechanism ensures the initial features of any two nodes can affect each other even if they are faraway neighbors, along with the network going deeper. We use $\mathbf{x}_v^l$ to denote the learned representation of node v from the l-th layer in a GNN ($l = 1, \cdots, L$). Specifically, $\mathbf{x}_v^0 = \mathbf{x}_v$. The output node features of the l-th layer, which also formulate the input to the next layer, are generated as follows:

$$\mathbf{x}_i^{l+1} = \sigma\Big(\sum_{v_j \in \mathcal{N}(i)} \frac{1}{\sqrt{d_i \cdot d_j}} \mathbf{W}^l \mathbf{x}_j^l \Big), \tag{1}$$

where $\mathcal{N}(i)$ denotes the immediate neighbor nodes of v_i and σ is the activation function (e.g., ReLU).

We take node classification as an example task for the rest of the paper, without loss of generality. The objective of training GNNs is to minimize the following cross-entropy loss function:

$$\mathcal{L} = \sum_{v_i \in \mathcal{V}^L} \mathcal{L}(v_i) = - \sum_{v_i \in \mathcal{V}^L} y_i \log \hat{y}_i, \tag{2}$$

where y_v and $\hat{y}_v$ are true and predicted labels, respectively. Typically, $\hat{y}_v = \text{Softmax}(\mathbf{x}_v^{(L)})$ is acquired by applying Softmax to the representations from the last layer.

4 DATA ANALYSIS

In this section, we conduct preliminary analysis on real-world graphs to show the properties of real-world graphs for semi-supervised node classification and the issue of GCNs on these datasets. The preliminary analysis lays a solid foundation and paves us a way to design better GCNs. Since we aim to discover the issue of GCNs on real-world datasets, we choose four widely used datasets from GCNs literature to perform the analysis, which includes Cora, Citeseer, Pubmed [15], and Reddit [10]. Note that the split of training, validation and testing on all datasets are the same as described in the cited papers.

4.1 Degree Distribution

The degree distribution of most real-world graphs follows the power-law [1, 9]. To verify this, we plot the degree distribution of the four datasets in Figure 1. As we can see from the figure, degrees

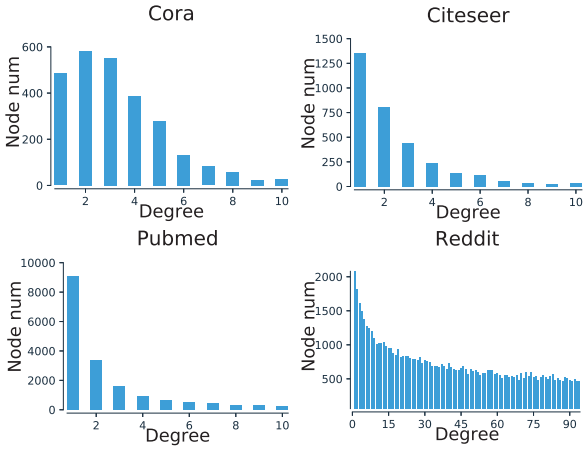


Figure 1: Degree distribution.

of the majority of nodes are relatively low, and decrease as the value of degree raise up. The shape of the degree distributions verify our assumption. The power-law distribution indicates nodes on graph are non-*i.i.d* distributed. Applying the same network parameters on all nodes may result in sub-optimal prediction/classification.

4.2 Accuracy Varying Node Degree

GCNs rely on message-passing mechanism, and aggregates the information from neighbors to learn representative embedding vectors. Because the degree of nodes follows a nonuniform (power-law) distribution, low-degree nodes, which are the majority, will receive less information during the aggregation. As a results, the error rate on low-degree nodes could be higher. To validate the assumption, we train GCNs following the same setting in [15], and report its error rate on node classification tasks w.r.t degree of nodes. From Figure 2, we find that, when degree is small, the error rate decreases significantly as the degree of nodes becomes larger. This verify our assumption that low-degree nodes receive less information during the aggregation and GCNs is biased against low-degree nodes.

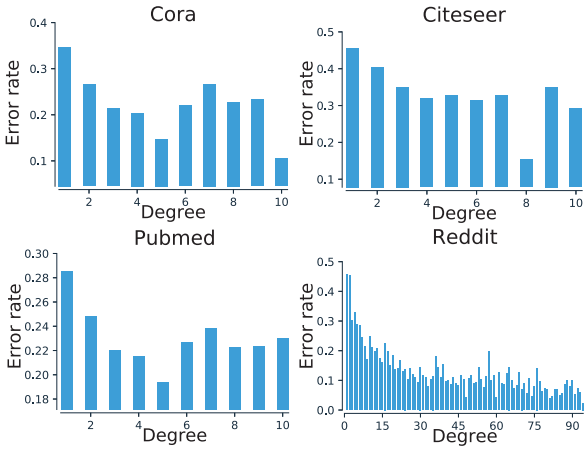


Figure 2: Error distribution w.r.t node degree.

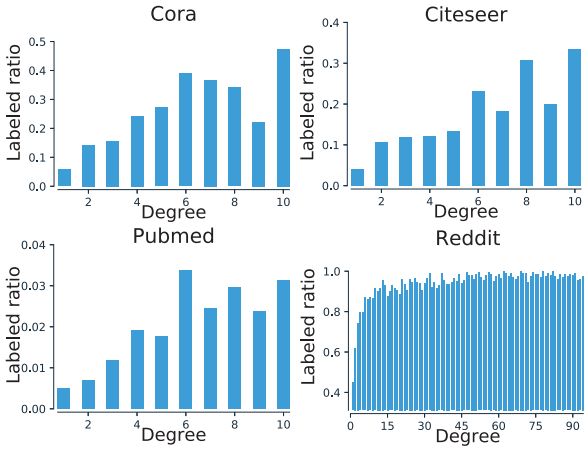


Figure 3: Ratio of being neighbor with a labeled node.

4.3 Labeled Neighbor Distribution

To further understand how the non-uniform degree distribution hurts GCNs, we analyze the probability of being connected to any labeled neighbor w.r.t node degree, as illustrated in Figure 3. We can conclude that nodes with higher degrees are much more likely to own labeled neighbors comparing with lower degree ones. In training process, GCNs use back-propagation to update its neural parameters such that the classification error on labeled nodes is reduced. Thanks to the message-passing mechanism, nodes with labeled neighbors participate more frequently in the optimization process. As a result, GCNs performs better on high-degree nodes.

4.4 Bridging Node Degree and Biases in GCNs

Inspired by Koh and Liang [16] and Xu et al. [35], we borrow ideas of sensitivity analysis and influence functions in statistics field to measure the influence of a specific node to the accuracy of GCNs. We first define node influence from node v_i to v_k as follows:

$$I(i,k) = \| \mathbb{E}(\partial \mathbf{x}_i^L / \partial \mathbf{x}_k) \|, \tag{3}$$

which measures how the feature of v_i affects the training of GCN on node v_k. Because the loss function is defined purely on labeled nodes, the influence of any unlabeled node (say v_i) to the whole GCN can be approximated by the overall influence of every labeled node:

$$S(i) = \sum_{v_k \in \mathcal{V}^L} I(i,k). \tag{4}$$

We can summarize the relation of node degree and the performance of GCNs in the following theorem:

Theorem 4.1. *Assume ReLU is the activation function. Let v_i and v_j denote two nodes in a graph. If we have $d_i > d_j$, then the influence score follows: $S(i) > S(j)$ of an untrained GCN.*

Proof. The partial differential between $\mathbf{x}_i^l$ and $\mathbf{x}_k$ is derived as:

$$\frac{\partial \mathbf{x}_i^l}{\partial \mathbf{x}_k} = \frac{1}{\sqrt{d_i}} \cdot \mathrm{diag}(\mathbb{1}_{\sigma_l}) \cdot \mathbf{W}^k \cdot \sum_{v_n \in \mathcal{N}(i)} \frac{1}{\sqrt{d_n}} \frac{\partial \mathbf{x}_n^{l-1}}{\partial \mathbf{x}_k}, \tag{5}$$

where σ_l denote the output from the activation function (i.e. ReLU) at the l-th GCN layer, and $\mathrm{diag}(\mathbb{1}_{\sigma_l})$ is a diagonal mask matrix representing the activation result. Using chain rule, we further derive:

$$\frac{\partial \mathbf{x}_i^L}{\partial \mathbf{x}_k} = \sqrt{d_i d_k} \cdot \sum_{p=1}^{\Psi} \prod_{l=L}^{0} \frac{1}{d_{p^l}} \mathrm{diag}(\mathbb{1}_{\sigma_l}) \cdot \mathbf{W}^l, \tag{6}$$

where Ψ is the set of all $(L+1)$-length random-walk paths on the graph from node v_i to v_k, and p^l represents the l-th node on a specific path p (p^L and p^0 denote node i and k, accordingly). Note that every path is fully-connected where $v_{p^l} \in \mathcal{N}(p^{l+1})$ for any p and any l. Similar to Xu et al. [35], the expectation of $\partial \mathbf{x}_i^L / \partial \mathbf{x}_k$ can be estimated as follows:

$$\begin{aligned} \mathbb{E}\left(\frac{\partial \mathbf{x}_i^L}{\partial \mathbf{x}_k}\right) &= \sqrt{d_i d_k} \cdot \sum_{p=1}^{\Psi} \mathbb{E}\left(\prod_{l=L}^{0} \frac{1}{d_{p^l}} \mathrm{diag}(\mathbb{1}_{\sigma_l}) \cdot \mathbf{W}^l\right) \\ &= \rho \sum_{v_n \in \mathcal{N}(i)} \sum_{p=1}^{\Psi_n} \mathbb{E}\left(\prod_{l=L-1}^{0} \frac{1}{d_{p^l}} \mathrm{diag}(\mathbb{1}_{\sigma_l}) \cdot \mathbf{W}^l\right), \end{aligned} \tag{7}$$

where $\rho = (\sqrt{d_k}/\sqrt{d_i}) \cdot \mathrm{diag}(\mathbb{1}_{\sigma_L}) \cdot \mathbf{W}^L$ only correlated to v_i and v_k, and Ψ_n denote the set of all L-length walks from a neighborhood of v_i to v_k. Assume the neighborhoods are randomly distributed (i.e., v_n is (near) randomly sampled), the expectation on walks starting from neighborhoods can be replaced by a constant value ν:

$$\sum_{p=1}^{\Psi_n} \mathbb{E}\left(\prod_{l=L-1}^{0} \frac{1}{d_{p^l}} \mathrm{diag}(\mathbb{1}_{\sigma_l}) \cdot \mathbf{W}^l\right) = \nu, \tag{8}$$

and we further have:

$$\mathbb{E}\left(\frac{\partial \mathbf{x}_i^L}{\partial \mathbf{x}_k}\right) = \rho d_i \nu = \nu \sqrt{d_k d_i} \cdot \mathrm{diag}(\mathbb{1}_{\sigma_L}) \cdot \mathbf{W}^L \propto \sqrt{d_i},$$

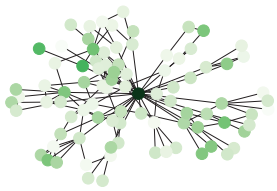
therefore, if $d_i > d_j$, then we have $\mathbb{E}\left(\frac{\partial \mathbf{x}_i^L}{\partial \mathbf{x}_k}\right) > \mathbb{E}\left(\frac{\partial \mathbf{x}_j^L}{\partial \mathbf{x}_k}\right)$. By summing up over all labeled nodes in $\mathcal{V}^L$, we have $S(i) > S(j)$. □

We validate our conclusion in Figure 4.

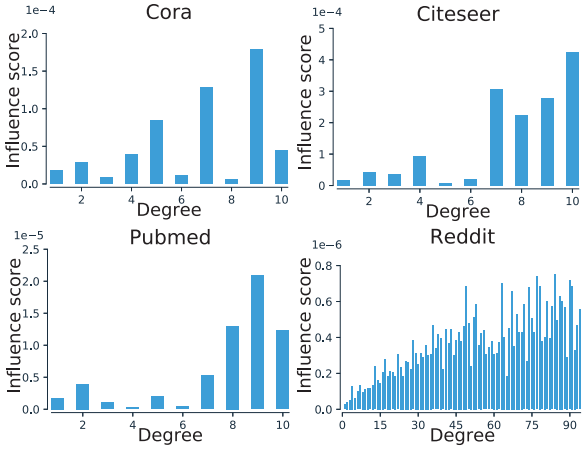
We first visualize the influence score distribution on a subgraph of the Cora dataset in Figure 4a. Clearly, the hub node at the center of the graph gains a much higher influence score than others. We further analyze the distribution of the influence score on four datasets, and report the results in Figure 4b. Clearly, the influence score increases as the node degree becomes larger. This indicates that nodes with larger degrees have higher impact on the training process of GCN, resulting in imbalanced error rate distribution over different degrees.

5 APPROACH

With the above analysis, we summarize the limitations of GCNs as follows: (1) GCNs use the same set of parameters for all nodes and fails to model the local intra- and inter- relations of nodes, resulting in model-aspect biases; (2) low degree nodes are less likely to have labeled neighbors and participate inactively when training GCNs, such biases come from the data distribution aspect. To address these issues, we propose SL-DSGCN that improves GCNs from two folds: *Firstly*, we propose a degree-specific GCN (DSGCN) layer whose parameters are generated by a recurrent neural network (RNN). Nodes with different degrees have their own specific parameters so that the local intra-relation is captured. Besides, as parameters



(a) Topology of influence score on a subgraph of Cora. Darker colors denote higher influences.



(b) Distribution of influence score varying node degree.

Figure 4: Distributions of the Influence Score.

are iteratively generated from the same RNN, their inner correlations help model the inter-relation of nodes with similar degrees. The DSGCN layer balances the global generalization and local discrepancies for nodes with various degrees. *Secondly*, we design a self-supervised-learning algorithm to construct pseudo labels with uncertainty within unlabeled nodes. This is achieved by training a Bayesian neural network (BNN). The DSGCN is fine-tuned on both true and pseudo labels, where the artificial ones are weighted according to their uncertainties. This prevents SL-DSGCN from overfiiting to inaccurate pseudo labels.

5.1 Degree-Specific GCN Layer

As the training of a GCN is dominated by high-degree nodes, using one set of parameters could lead to sub-optimal results. To increase the diversity of learned parameters for nodes with different degrees, following aggregation can be used to distinguish the degree-specific information from the graph:

$$\mathbf{x}_i^{l+1} = \sigma\Big(\sum_{j \in \mathcal{N}(i)} a_{ij} (\mathbf{W}^l + \mathbf{W}^l_{d(j)}) \mathbf{x}_j^l \Big), \tag{9}$$

where $\mathbf{W}^l_{d(j)}$ captures degree-specific information. $\mathbf{W}^l$ is the original GNN parameters at layer l in Eqn 1.

The design of $\mathbf{W}^l_{d(j)}$ is a non-trivial task. One straight-forward way is making degree-specific parameters unique for all degrees. However, the maximum value of node degree on a graph can be extremely large due to the long-tail power-law distribution, constructing unique parameters for every degree is impractical. Besides, some higher degrees are underrepresented, with only few nodes available. How to prevent underfitting issue for them is also a challenging problem. To overcome this issue, Wu et al. [31] propose

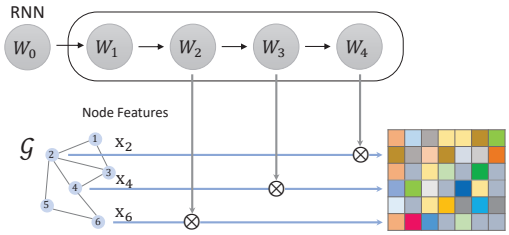


Figure 5: GNN with degree-specific trainable parameters. Node features multiply with different parameters generated by the RNN according to their degree.

a hashing-based solution where some degrees are mapped to the same entry of a hash table containing multiple sets of GCN parameters. By manually tuning the size of the hash table, the total number of degree-specific parameters is under control.

However, the hashing-based approach randomly maps node degree to parameters, and ignores the local inter-relations of nodes with similar degrees. If two nodes have close degree values, their may have a tight correlation. The necessity of capturing local inter-relation of nodes motivates us to adopt an RNN to generate the degree-specific parameters, which is shown in Figure 5. Specifically, let $\mathbf{W}^l_0$ denote the initialization input to an RNN cell RNN$(\cdot)$, degree-specific parameters are generated as follows:

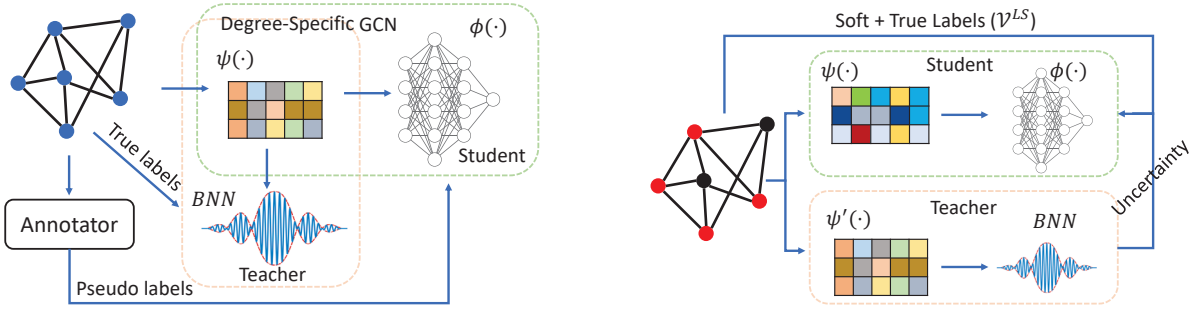
$$W^l_{k+1} = \text{RNN}(W^l_k), \ k = 0, 1, \cdots, d_{\max}, \tag{10}$$

where W^l_{k+1} is the updated hidden state of the RNN after feeding W^l_k as the input, and $d_{\max}$ is a threshold to prevent long-tail issue of the degrees. Nodes with degree higher than $d_{\max}$ are processed using $W^l_{\max+1}$. The generated parameters can cover every degree. The advantages of using an RNN are (1) as RNN is iterating over all degrees, generated degree-specified parameters are correlated with each other corresponding to the degree so that modeling local inter-relations of nodes is guaranteed; (2) the total number of actual trainable parameters is fixed (i.e., the initialization input and parameters in the RNN cell), which is more efficient comparing with setting up every $\mathbf{W}^l_{d(i)}$ separately or use a hashing table. Note that the generated parameters from RNN naturally capture the local intra-relation because every degree has its unique parameters. Besides, the shared parameters $\mathbf{W}^l$ handles the globally shared node relations.

While the DSGCN layer reduces degree-related biases in GCNs from the model aspect, low-degree nodes still participate less frequently when training the DSGCN. To provide sufficient supervisions for low-degree nodes, we introduce a self-supervised-learning algorithm that creates high-quality pseudo-labels on unlabeled nodes.

5.2 Self-Supervised-Training with Bayesian Teacher Network

In most semi-supervised settings on graph data, the number of unlabeled nodes is much larger than that of labeled ones (i.e., $|\mathcal{V}^L| \ll |\mathcal{V}^U|$. We assume the existence of a graph annotator that can heuristically generate pseudo-labels for nodes in $\mathcal{V}^U$, such as propagation algorithm [41], label spreading [40], and PairWalks [32]. The pseudo-labels are noisy and less accurate compared with the true labels from $\mathcal{V}^L$ because of the limitations of the annotator.



(a) Pre-train student and teacher. **(b) Finetune the student on $\mathcal{V}^{LS}$ with dynamic step size.**

Figure 6: Overall framework of SL-DSGCN.

The intuition of proposed self-learning algorithm is to leverage the large amount of pseudo-labels in the training of GCNs so that even for low-degree nodes can have labeled neighbors. However, different from existing literature [17, 24] that use pseudo labels in the same way of true labeled nodes, we also judge the quality of pseudo labels to avoid overfitting on inaccurate pseudo labels. Specifically, we design a Bayesian neural network as a *teacher* to justify the quality of pseudo-labels from the annotator, so that the GCNs as a *student* can fully exploit the pseudo-labels. There are two steps of the self-learning process as illustrated in Figure 6.

5.2.1 Pre-training with the Annotator. Firstly, we build the student network using the proposed degree-specific GNN layer. As shown in Figure 6a, the student first applies multiple DSGCN layers over the input graph ($\psi(\cdot)$ part) to capture the dependencies of graph structure and to model the correlation among nodes with different degrees. Taking the graph $\mathcal{G}$ as an input, $\psi(\cdot)$ transform each node into its representation vector. To further classify each node, we then apply fully-connected layers followed by a softmax layer ($\phi(\cdot)$ part) on representation vectors from $\psi(\cdot)$. Different from conventional GNNs, the student network leverage $\psi(\cdot)$ to learn data representation from the graph, and assign the classification task to the second part $\phi(\cdot)$. Using the pseudo labels from the annotator, we pre-train the student network so that $\psi(\cdot)$ is fitted to the data and $\phi(\cdot)$ becomes a noisy classifier. The whole student network is represented by $\phi(\psi(\cdot))$.

However, simply treating all pseudo labels as ground truth will hurt the performance. We then design a teacher network to estimate the uncertainty of pseudo labels from the annotator. The teacher network is constructed based on a Bayesian neural network (BNN) [**?**]. We use the node representation from the data representation learner $\psi(\cdot)$ as the input, to train a fully-connected BNN using real-world truely labeled nodes $\mathcal{V}^L$, as illustrated in Figure 6a. In particular, the BNN aims at learning the posterior distribution of its parameters, defined as follows:

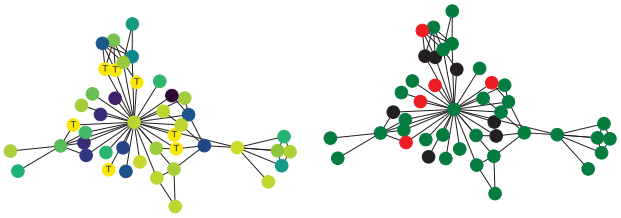
$$p(\zeta|\psi(x)) \propto p(\psi(x)|\zeta) \cdot p(\zeta), \tag{11}$$

where ζ denotes the parameters of the BNN, $p(\zeta)$ is the prior of ζ that contains our assumption of the network parameters, and $p(\psi(x)|\zeta)$ is the likelihood which describe the input data (i.e., node representation from $\psi(x)$). The probability distributions of model parameters ζ are updated with the Bayes theorem taking into account both the prior and the likelihood. Without loss of generalities, we use normal distribution as the prior for the BNN. We fix the representation learner when updating the BNN part, so that the teacher can leverage the knowledge from the annotated results. Besides, training on top of $\psi(\cdot)$ ensures the teacher is learning in the same representation space of the student, so that the judgement of unlabeled nodes in further steps is unbiased and has no domain shifting for the student network. We use a two-layer fully-connected network as the approximation for the likelihood. The posterior mean μ and posterior covariance κ of the BNN is acquired after training the BNN model, and are further used to create soft labels on unlabeled nodes with uncertainties. In particular, for every unlabeled node $v_i \in \mathcal{V}^U$, we acquire its prediction and uncertainty score as follows:

$$y_i^s = f(\mu(x_i)), \quad c_i = g(\kappa(x_i)),$$

where $f(\cdot)$ and $g(\cdot)$ are two functions (e.g., neural networks) that map the posterior mean and covariance vectors to desired soft label and uncertainty score.

We visualize the prediction and uncertainty of the teacher BNN trained on a small subset from the reddit network dataset in Figure 7. As we can see in Figure 7a, the uncertainty for labeled nodes are almost zero, indicating the teacher fit the training data very well. Meanwhile, we also observe that the uncertainty scores on low degree nodes tend to be larger, which is consistent with our previous analysis. As low degree nodes have less impact on the training loss function and receive less supervision from labeled neighbors, it is harder to generate a confident prediction for them. Similarly in Figure 7b, it is more likely for low degree nodes to be misclassified than high degree ones.



(a) Uncertainty scores from the teacher network. Darker color means higher uncertainty and "⊤" denotes training nodes. **(b) Classification error of the teacher network. Red and green denote wrong and correct prediction respectively, and black represents training nodes.**

Figure 7: Uncertainty score and error distribution of the teacher network. Generally, nodes closer to labeled (training) ones tend to have lower uncertainty and error rate.

5.2.2 Fine-tuning Student with Uncertainty Scores. After the pre-training of student and teacher network, the second step of the self-learning process is fine-tuning the student network using generated labels and uncertainty scores from the teacher. We define a softly-labeled node set $\mathcal{V}^S \subset \mathcal{V}^U$ where nodes in $\mathcal{V}^S$ are labeled identically by both the annotator and the teacher. The intuition is similar to majority vote. Given large amount of unlabeled nodes, it is worthwhile to compile a cleaner labeled node set as a compensation to the existing true labeled nodes.

Existing works exploring self-learning for GNNs treat selected pseudo labels in the same way of using labeled nodes. For example, Li et al. [17] and Sun et al. [24] progressively add selected nodes with pseudo labels into the training set. However, such solutions are sub-optimal. One bottleneck is that all selected pseudo labels are equally treated, and are utilized in the same way of true labeled nodes. However, even for pseudo labels with high confidence, they still contain more noise than the real labeled part.

Fortunately, the proposed BNN-based teacher network naturally solves the above challenge. The generated uncertainty scores can be utilized when training with pseudo labels. Specifically, we fine-tune the student network on $\mathcal{V}^{LS} = \mathcal{V}^L \cup \mathcal{V}^S$ using stochastic gradient descent (SGD) algorithm, where the uncertainty score controls the step size for each nodes in $\mathcal{V}^{LS}$. We use θ to denote parameters in the student network, the optimization (learning) goal is as follows:

$$\theta^* = \text{argmax}_\theta \mathcal{L}(\theta) = \sum_{v_i \in \mathcal{V}^{LS}} \mathcal{L}(v_i; \theta). \tag{12}$$

The updating rule for parameters θ is:

$$\theta' = \theta - \sum_{v_i \in \mathcal{V}^{LS}} \eta_i \mathcal{L}(v_i; \theta), \tag{13}$$

where η_i is a dynamic step size defined as follows:

$$\eta_i = \eta \cdot \eta_i^c \cdot \eta_i^d = \eta \cdot \exp(-\alpha c_i) \cdot \exp(\beta d_i), \tag{14}$$

which contains three parts. The first part η is the global step size used in classic SGD. The second part η_i^c penalize each sample (node) by its quality, using the uncertainty score acquired from the teacher network. We choose a negative exponential function over the uncertainty score so that nodes with larger uncertainty participate less in the updating process. The third term empirically assigns larger weights to nodes with higher degrees according to the observations in Figure 4a and Figure 7. Here α and β are hyperparameters that balance three parts in the dynamic step size. Generally, larger values of α and/or β pay more attention to the uncertainty scores and the degree distribution, correspondingly. After fine-tining on $\mathcal{V}^{LS}$ using SGD with dynamic step size, we use the student network to predict node labels.

5.3 Training Algorithm

We summarize the self-learning process in Algorithm 1. Line 1-3 are the pre-training of student and teacher network. After acquiring predictions and uncertainty scores from the pre-trained teacher in Line 4, we compile $\mathcal{V}^{LS}$ using true labels and the softly-labeled nodes (Line 5-6). Finally, as introduced in Line 7-9, the student network is fine-tuned on $\mathcal{V}^{LS}$ with dynamic step size. Note that although we select GCN as the basis of SL-DSGCN, the idea of capturing globally shared, local intra- and inter- relations of nodes with an RNN-based parameter generator, and using self-supervised-learning with dynamic step size are model agnostic. Namely, they can also be applied on other GNN models, such as graph attention networks [28], GraphSAGE [10], etc. We leave this part for future work.

Algorithm 1: Self-learning for SL-DSGCN

Input: $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X})$
Output: Parameters θ of student network $\phi(\psi(\cdot))$
// Pre-training
1 Acquire pseudo-labels for $\mathcal{V}^U$ using a graph annotator;
2 Pre-train $\phi(\psi(\cdot))$ on pseudo labels;
3 Fix $\psi(\cdot)$ and pre-train BNN part of the teacher network;
4 Acquire prediction y_i^s and uncertainty score c_i for every node in $\mathcal{V}^U$ from the teacher;
// Fine-tuning
5 Compile a soft-labeled node set $\mathcal{V}^S \subset \mathcal{V}^U$ where the teacher network agrees with the annotator;
6 Build $\mathcal{V}^{LS} = \mathcal{V}^L \cup \mathcal{V}^S$ to fine-tune the student network;
7 **while** *not converge* **do**
8 Compute dynamic step size η_i for $v_i \in \mathcal{V}^{LS}$ as $\eta_i = \eta \cdot \eta_i^c \cdot \eta_i^d$;
9 Update parameters of the student network as $\theta' = \theta - \sum_{v_i \in \mathcal{V}^{LS}} \eta_i \mathcal{L}(v_i; \theta)$;
10 **end**

6 EXPERIMENTS

In this section, we conduct experiments on real-world datasets to evaluate the effectiveness of SL-DSGCN. In particular, we aim to answer the following questions:

- Can SL-DSGCN outperform existing self-training algorithms for GNNs on various benchmark datasets?
- How do the degree-specific design (DSGCN), the machine teaching approach, and the dynamic step size contribute to SL-DSGCN?
- How sensitive of SL-DSGCN is on the selection of softly-labeled node set?

Next, we start by introducing the experimental settings followed by experiments on node classification to answer these questions.

6.1 Experimental Setup

6.1.1 Datasets. For a fair comparison, we adopt same benchmark datasets used by Sun et al. [24] and Li et al. [17], including Cora, Citeseer, Pubmed [22]. Each dataset contains a citation graph, where nodes represent articles/papers and edges denote citation correlation. Node features are constructed using bag-of words features. The detailed statistics of the datasets are summarized in Table 1.

Table 1: Statistics of the Datasets

Dataset	Nodes	Edges	Classes	Features
Cora	2708	5429	7	1433
CiteSeer	3327	4732	6	3703
PubMed	19717	44338	3	500

Table 2: Node Classification Performance Comparison on Cora, Citseer and PubMed

Dataset	Cora					Citeseer					PubMed		
Label Rate	0.5%	1%	2%	3%	4%	0.5%	1%	2%	3%	4%	0.03%	0.06%	0.09%
LP	29.05	38.63	53.26	70.31	73.47	32.10	40.08	42.83	45.32	49.01	39.01	48.7	56.73
ParWalks	37.01	41.40	50.84	58.24	63.78	19.66	23.70	29.17	35.61	42.65	35.15	40.27	51.33
GCN	35.89	46.00	60.00	71.15	75.68	34.50	43.94	54.42	56.22	58.71	47.97	56.68	63.26
DEMO-Net	33.56	40.05	61.18	72.80	77.11	36.18	43.35	53.38	56.5	59.85	48.15	57.24	62.95
Self-Train	43.83	52.45	63.36	70.62	77.37	42.60	46.79	52.92	58.37	60.42	57.67	61.84	64.73
Co-Train	40.99	52.08	64.27	73.04	75.86	40.98	56.51	52.40	57.86	62.83	53.15	59.63	65.50
Union	45.86	53.59	64.86	73.28	77.41	45.82	54.38	55.98	60.41	59.84	58.77	60.61	67.57
Interesction	33.38	49.26	62.58	70.64	77.74	36.23	55.80	56.11	58.74	62.96	59.70	60.21	63.97
M3S	50.28	58.74	68.04	75.09	78.80	48.96	53.25	58.34	61.95	63.03	59.31	65.25	70.75
SL-DSGCN	**53.58**	**61.36**	**70.31**	**80.15**	**81.05**	**54.07**	**56.68**	**59.93**	**62.20**	**64.45**	**61.15**	**65.68**	**71.78**

6.1.2 Baselines. We compare SL-DSGCN with representative and state-of-the-art node classification algorithms, which includes:

- LP [41]: Label Propagation is a classical self-supervised learning algorithm which where we iteratively assign labels to unlabelled points by propagating labels through the graph. It serves as the weak annotator in our framework.
- ParWalks [32]: ParWalks extends label propagation by using partially absorbing random walk.
- GCN [15]: GCN is a widely used graph neural network. It defines graph convolution via spectral analysis.
- DEMO-Net [31]: It proposes multi-task graph convolution where each task represents node representation learning for nodes with a specific degree value, thus leading to preserving the degree specific graph structure. DEMO-net also contains other constraints to improve the representation learning, including order-free and seed-oriented. These constraints are removed for a fair comparison because they do not tackle the degree-related biases of GCNs, and can be applied on all above methods. We choose the weight version of DEMO-net due to better performances.
- Co-Training [17]: This method uses the ParWalk to find the most confident vertices – the nearest neighbors to the labeled vertices of each class, and then add them to the label set to train a GCN.
- Self-Training, Union and Intersection [17]: Self-training picks the most confident soft-labels of GCN and puts it into the labeled node set to improve the performance of GCN. Union takes the union of the most confident soft-labels by both GCN and ParWalk as self-supervision while Intersection takes the intersection of the two as the self-supervision.
- M3S [24]: Multi-Stage Self-Supervised Training leverages DeepCluster technique to provide self-supervision and utilizes the cluster information to iterative train GNN.

6.1.3 Settings and Hyperparameters. The training and testing set are generated as follows: we randomly sample x% of nodes for training, 35% nodes for testing, and treat the remained nodes as unlabeled ones for each dataset. Furthermore, to understand how SL-DSGCN performs under various label sparsity scenarios in real-world, for CORA and Citeseer, we vary x as $\{0.5, 1, 2, 3, 4\}$. Since PubMed is relative larger than Cora and CiteSeer, we vary x as $\{0.03, 0.06, 0.09\}$ for it. Note that we set x as small values because in typical setting of real-world semi-supervised node classification tasks, only a small amount of nodes are labeled for training [17, 24]. We adopt the same hyper-parameters for GCN as introduced by Kipf and Welling [15], which is a two-layer GCN with 16 hidden units on each layer. For DEMO-Net, Self-Train, Co-train, Union, and Intersection, we adopt their public code and tune hyperparameters for the best performance. We implement M3S following the descriptions in the paper [24]. For the student network part, both $\phi(\cdot)$ and $\psi(\cdot)$ are implemented by one DSGCN layer. We set $d_{\max}$ to 10. The Bayesian neural network part of the teacher contains two fully-connected layers, each contains 16 hidden units. We fix α and β to 1. Note that for fair comparison, we set all self-supervised-learning GCNs to two-layers with 16 hidden units, which is aligned with both GCN and SL-DSGCN. We report the averaged results over 10 times of running.

6.2 Node Classification Performance

To answer the first research question, we conduct node classification with comparison to existing self-training algorithms for GNNs on the datasets introduced above. The experimental results in terms of accuracy for the three datasets are reported in Table 2. From the table, we make the following observations:

- Generally, self-supervision based approaches such as M3S, Intersection and Union outperform algorithms without self-supervision such as LP and GCN, which implies that self-supervision could help provide more labeled nodes to training so that the percentage of labeled neighborhood of low-degree increases.
- As label rate x increases, the performance improvement of self-supervision based approaches over non-self-supervision approaches decreases. For example, on Cora dataset, as x increase from 0.5% to 4%, the performance improvement of M3S and SL-DGNN over GCN are $\{14.39, 12.74, 8.04, 3.94, 3.12\}$ and $\{17.69, 15.36, 10.31, 9.00, 5.37\}$, respectively. This is because as the amount of labeled data increases, the percentage of labeled neighborhood of low-degree also increases, which makes the introduction of self-supervision less useful.
- For all the three datasets and label rate, SL-DSGCN consistently outperforms all the baselines significantly, which shows the effectiveness of the proposed framework. In particular, both M3S and SL-DSGCN adopt self-supervision. SL-DSGCN significantly outperforms M3S because SL-DSGCN explicitly model degree-specific GNN layer through LSTM, which could benefit the low-degree nodes more.

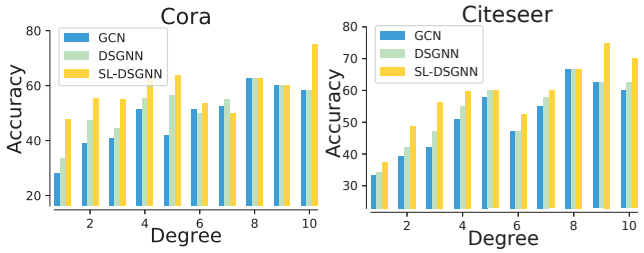


Figure 8: Node Classification Performance on Nodes with Different Degrees

6.3 Performance on Low Degree Nodes

SL-DSGCN is motivated by the observation that the number of labeled nodes for low-degree nodes is very much smaller than that of high-degree nodes, which makes GNN biased towards high-degree nodes. Thus, degree specific GNN layer and self-training with Bayesian teacher networks are leveraged to alleviate the issue. To validate the effectiveness of the proposed framework SL-DSGCN on low-degree nodes, we further visualize the node classification performance of low-degree nodes on Cora and Citeseer in Figure 8. Note that for Cora and Citeseer, 96.45% and 97.53% nodes have a degree less than 11. From the figure, we observe that:

- Both DSGCN and SL-DSGCN outperform GNN significantly, especially on node with small degrees, which shows the effectiveness of degree specific layer and self-supervision for improving performance of low-degree nodes. In addition, SL-DSGCN has better performance than DSGCN, which implies that the degree specific layer and self-supervision improves the performance from two different perspectives. Degree specific layer tries to learn node-specific parameters to reduce the bias towards high-degree nodes while self-supervision tries to improve the number labeled nodes in each node's neighborhood.
- When degree the node degree is very small, say $\{1, 2, 3, 4, 5\}$, the improvement of DSGCN and SL-DSGCN is very significant. As the degree become larger, the improvement becomes smaller. This is because when degree is very small, most of these nodes have very few labeled nodes in their neighborhood. A small amount of soft-label and the degree-specific parameters could improve the performance a lot. However, when the degree become larger, there are already enough supervision to train a good GNN, which makes the improvement insignificant. However, as the majority nodes in graphs are low degree nodes, SL-DSGCN can still improve the overall performance significantly.

6.4 Ablation Study

In this subsection, we conduct ablation study to understand the impact of degree-specific GNN, the dynamic step size for SGD, and the self-teaching algorithm, which answers the second research question. Specifically, several variations of SL-DSGCN are compared including (1): DSGCN which applies the degree-specific parameters on GCN; (2) MT-GNN which replace the dynamic step size with original one and remove the softly-labeled node set from $\mathcal{V}^{LS}$ (i.e., only use the labeled nodes for fine-tuning the student network). MT-GNN can be treated as a GNN enhanced by the vanilla machine teaching algorithm; (3) SL-DSGCN$_{fs}$ which removes the dynamic step size; and (4) SL-GNN which removes the degree-specific design in the student network. The performance of SL-DSGCN and the variants on Cora and Citeseer are reported in Table 3 and 4, respectively. From these two tables, we observe that: (i) In terms of the comparison between SL-GNN and SL-DSGCN, SL-DSGCN performs slightly better than SL-GNN, which shows that degree specific layer can slightly improve the performance; (ii) In terms of the comparison between SL-DSGCN$_{fs}$ and SL-DSGCN, SL-DSGCN has better performance than SL-DSGCN$_{fs}$, which is because SL-DSGCN$_{fs}$ doesn't adopt the dynamic step size; and (iii) SL-DSGCN significantly outperforms DSGCN, which shows the effectiveness of the proposed self-supervised training.

Table 3: Ablation study on Cora dataset.

Label Rate	0.5%	1%	2%	3%	4%
DSGCN	36.11	47.67	61.91	73.87	77.03
MT-GNN	50.51	57.47	67.26	78.52	78.84
SL-DSGCN$_{fs}$	51.36	59.85	68.81	79.14	79.90
SL-GNN	52.05	60.41	69.51	79.75	80.21
SL-DSGCN	53.58	61.36	70.31	80.15	81.05

Table 4: Ablation study on Citeseer dataset.

Label Rate	0.5%	1%	2%	3%	4%
DSGCN	37.51	44.75	55.41	56.9	60.24
MT-GNN	49.78	50.75	55.14	59.01	61.23
SL-DSGCN$_{fs}$	51.89	53.26	58.38	60.63	62.15
SL-GNN	52.77	54.79	57.27	61.98	63.99
SL-DSGCN	54.07	56.68	59.93	62.20	64.45

Table 5: Influence of the softly-labeled node set.

Dataset	Node set	0.5%	1%	2%	3%	4%
Cora	DSGCN	36.11	47.67	61.91	73.87	77.03
	$\mathcal{V}_A^S$	47.21	55.10	67.15	76.39	75.07
	$\mathcal{V}_T^S$	50.73	58.29	68.85	77.24	76.93
	SL-DSGCN	53.58	61.36	70.31	80.15	81.05
Citeseer	DSGCN	37.51	44.75	55.41	56.9	60.24
	$\mathcal{V}_A^S$	50.68	53.42	57.10	60.52	60.63
	$\mathcal{V}_T^S$	52.25	52.80	55.13	61.82	61.01
	SL-DSGCN	54.07	56.68	59.93	62.20	64.45

6.5 Sensitivity on Softly-labeled Node Set

In this subsection, we further analyze how the construction of softly-labeled node set can impact the performance of SL-DSGCN. We compare the intersection approach in SL-DSGCN with the following alternations: (1) using pseudo labels from the annotator and build $\mathcal{V}_A^S$ for all unlabeled nodes; (2) using predictions from the teacher network and compile $\mathcal{V}_T^S$ for all unlabeled nodes; and (3) without adding any soft labels, which is actually DSGCN. The node classification performance of SL-DSGCN with comparison to the three alternatives is reported in Table 5. From the table, we make the following observations: (i) Compared with training without soft-labels, i.e., trained on $\mathcal{V}^L$ only, using soft-labels, i.e., $\mathcal{V}_A^S$, $\mathcal{V}_T^S$ and $\mathcal{V}^S$, can significantly improve the performance, which shows the importance of soft-labels in providing supervision to GNN for classification; and (ii) Though $\mathcal{V}_A^S$, $\mathcal{V}_T^S$ and $\mathcal{V}^S$ all utilize

soft-labels, the performance of $\mathcal{V}^S$ is much better than $\mathcal{V}_A^S$ and $\mathcal{V}_T^S$, which indicates that the teacher network and the annotator may infer some wrongly labeled nodes that could negatively affect the performance. Taking the intersection of these two can help pick nodes with correct soft labels and improve the performance.

7 CONCLUSION

In this paper, we empirically analyze an issue of GNN for semi-supervised node classification, i.e., when labeled nodes are randomly distributed on the graph, nodes with low degrees tend to have very few labeled nodes, which results in sub-optimal performance on low-degree nodes. To solve this issue, we propose a novel framework SL-DSGCN, which leverages degree-specific GCN layers and the self-supervised-learning with Bayesian teacher network to introduce more labeled neighbors for low-degree nodes. Experimental results on real-world detests demonstrate the effectiveness of the proposed framework for semi-supervised node classification. Further experiments are conducted to help understand the contributions of each components of SL-DSGCN.

There are several interesting directions which need further investigation. First, the proposed DSGCN layer and self-supervised-learning with Bayesian teacher network are generic framework which can benefit various GNNs. In this paper, we only use GCN as backbone. We will investigate the framework for other GNNs such as GAT [28]. Second, we mainly focus on the degree issue of attributed graphs. Heterogeneous information networks [23] are also pervasive in the real world. Similar issue also exists in heterogeneous graphs. Therefore, we will extend SL-DSGCN for heterogeneously network by considering different types of links/edges.

ACKNOWLEDGEMENT

This material is based upon work supported by, or in part by, the National Science Foundation (NSF) under grant IIS-1909702, IIS-1955851, and the Global Research Outreach program of Samsung Advanced Institute of Technology under grant #225003. The findings and conclusions in this paper do not necessarily reflect the view of the funding agency.

REFERENCES

[1] Réka Albert and Albert-László Barabási. 2002. Statistical mechanics of complex networks. *Reviews of modern physics* (2002).
[2] James Atwood and Don Towsley. 2016. Diffusion-convolutional neural networks. In *Advances in neural information processing systems*.
[3] Michael M Bronstein, Joan Bruna, Yann LeCun, Arthur Szlam, and Pierre Vandergheynst. 2017. Geometric deep learning: going beyond euclidean data. *IEEE Signal Processing Magazine* (2017).
[4] Joan Bruna, Wojciech Zaremba, Arthur Szlam, and Yann LeCun. 2013. Spectral networks and locally connected networks on graphs. *arXiv:1312.6203* (2013).
[5] Mathilde Caron, Piotr Bojanowski, Armand Joulin, and Matthijs Douze. 2018. Deep clustering for unsupervised learning of visual features. In *ECCV*.
[6] Aaron Clauset, Cosma Rohilla Shalizi, and Mark EJ Newman. 2009. Power-law distributions in empirical data. *SIAM review* (2009).
[7] Hanjun Dai, Hui Li, Tian Tian, Xin Huang, Lin Wang, Jun Zhu, and Le Song. 2019. Adversarial attack on graph structured data. In *ICML*.
[8] Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. 2016. Convolutional neural networks on graphs with fast localized spectral filtering. In *NeurIPS*.
[9] Michalis Faloutsos, Petros Faloutsos, and Christos Faloutsos. 1999. On power-law relationships of the internet topology. *ACM SIGCOMM computer communication review* (1999).
[10] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. In *NeurIPS*.
[11] Chao Huang, Xian Wu, Xuchao Zhang, Chuxu Zhang, Jiashu Zhao, Dawei Yin, and Nitesh V Chawla. 2019. Online purchase prediction via multi-scale modeling of behavior dynamics. In *KDD*.
[12] Wei Jin, Tyler Derr, Haochen Liu, Yiqi Wang, Suhang Wang, Zitao Liu, and Jiliang Tang. 2020. Self-supervised Learning on Graphs: Deep Insights and New Direction. *arXiv preprint arXiv:2006.10141* (2020).
[13] Wei Jin, Yao Ma, Xiaorui Liu, Xianfeng Tang, Suhang Wang, and Jiliang Tang. 2020. Graph Structure Learning for Robust Graph Neural Networks. *arXiv:2005.10203* (2020).
[14] Longlong Jing and Yingli Tian. 2020. Self-supervised visual feature learning with deep neural networks: A survey. *T-PAMI* (2020).
[15] Thomas N Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *ICLR*.
[16] Pang Wei Koh and Percy Liang. 2017. Understanding black-box predictions via influence functions. In *ICML*.
[17] Qimai Li, Zhichao Han, and Xiao-Ming Wu. 2018. Deeper insights into graph convolutional networks for semi-supervised learning. In *AAAI*.
[18] Ruirui Li, Xian Wu, Xian Wu, and Wei Wang. 2020. Few-Shot Learning for New User Recommendation in Location-based Social Networks. In *WWW*.
[19] Alan Mislove, Massimiliano Marcon, Krishna P Gummadi, Peter Druschel, and Bobby Bhattacharjee. 2007. Measurement and analysis of online social networks. In *SIGCOMM*.
[20] Mehdi Noroozi and Paolo Favaro. 2016. Unsupervised learning of visual representations by solving jigsaw puzzles. In *ECCV*.
[21] Deepak Pathak, Philipp Krahenbuhl, Jeff Donahue, Trevor Darrell, and Alexei A Efros. 2016. Context encoders: Feature learning by inpainting. In *CVPR*.
[22] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Galligher, and Tina Eliassi-Rad. 2008. Collective classification in network data. *AI magazine* (2008).
[23] Chuan Shi, Yitong Li, Jiawei Zhang, Yizhou Sun, and S Yu Philip. 2016. A survey of heterogeneous information network analysis. *TKDE* (2016).
[24] Ke Sun, Zhouchen Lin, and Zhanxing Zhu. 2019. Multi-Stage Self-Supervised Learning for Graph Convolutional Networks on Graphs with Few Labels. *arXiv:1902.11038* (2019).
[25] Yiwei Sun, Suhang Wang, Xianfeng Tang, Tsung-Yu Hsieh, and Vasant Honavar. 2020. Adversarial Attacks on Graph Neural Networks via Node Injections: A Hierarchical Reinforcement Learning Approach. In *WWW*. 673–683.
[26] Xianfeng Tang, Yandong Li, Yiwei Sun, Huaxiu Yao, Prasenjit Mitra, and Suhang Wang. 2020. Transferring Robustness for Graph Neural Network Against Poisoning Attacks. In *WSDM*.
[27] Xianfeng Tang, Yozen Liu, Neil Shah, Xiaolin Shi, Prasenjit Mitra, and Suhang Wang. 2020. Knowing your FATE: Friendship, Action and Temporal Explanations for User Engagement Prediction on Social Apps. *arXiv:2006.06427* (2020).
[28] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv:1710.10903* (2017).
[29] Daixin Wang, Jianbin Lin, Peng Cui, Quanhui Jia, Zhen Wang, Yanming Fang, Quan Yu, Jun Zhou, Shuang Yang, and Yuan Qi. 2019. A Semi-supervised Graph Attentive Network for Financial Fraud Detection. In *ICDM*.
[30] Xiaoyang Wang, Yao Ma, Yiqi Wang, Wei Jin, Xin Wang, Jiliang Tang, Caiyan Jia, and Jian Yu. 2020. Traffic Flow Prediction via Spatial Temporal Graph Neural Network. In *WWW*.
[31] Jun Wu, Jingrui He, and Jiejun Xu. 2019. Demo-net: Degree-specific graph neural networks for node and graph classification. In *KDD*.
[32] Xiao-Ming Wu, Zhenguo Li, Anthony M So, John Wright, and Shih-Fu Chang. 2012. Learning with partially absorbing random walks. In *NeurIPS*.
[33] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and S Yu Philip. 2020. A comprehensive survey on graph neural networks. *IEEE Neural Networks and Learning Systems* (2020).
[34] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2018. How powerful are graph neural networks? *arXiv:1810.00826* (2018).
[35] Keyulu Xu, Chengtao Li, Yonglong Tian, Tomohiro Sonobe, Ken-ichi Kawarabayashi, and Stefanie Jegelka. 2018. Representation learning on graphs with jumping knowledge networks. *arXiv:1806.03536* (2018).
[36] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L Hamilton, and Jure Leskovec. 2018. Graph convolutional neural networks for web-scale recommender systems. In *KDD*.
[37] Jiaxuan You, Bowen Liu, Zhitao Ying, Vijay Pande, and Jure Leskovec. 2018. Graph convolutional policy network for goal-directed molecular graph generation. In *NeurIPS*.
[38] Bing Yu, Haoteng Yin, and Zhanxing Zhu. 2017. Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting. *arXiv:1709.04875* (2017).
[39] Jiani Zhang, Xingjian Shi, Junyuan Xie, Hao Ma, Irwin King, and Dit-Yan Yeung. 2018. Gaan: Gated attention networks for learning on large and spatiotemporal graphs. *arXiv:1803.07294* (2018).
[40] Dengyong Zhou, Olivier Bousquet, Thomas N Lal, Jason Weston, and Bernhard Schölkopf. 2004. Learning with local and global consistency. In *NeurIPS*.
[41] Xiaojin Zhu and Zoubin Ghahramani. 2002. Learning from labeled and unlabeled data with label propagation. (2002).
[42] Daniel Zügner, Amir Akbarnejad, and Stephan Günnemann. 2018. Adversarial attacks on neural networks for graph data. In *KDD*.